Reviewer Pack — Minimal Order of Kneser Graphs and Communication Complexity …

Entry c2792048ca24 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 11:42:06 UTC

Minimal Order of Kneser Graphs and Communication Complexity Rank Correlation

Entry ID: `c2792048ca24` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 11:42:06 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Design Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every n-vertex graph G, the minimal order of a Kneser graph containing G as an induced subgraph is linearly correlated with its communication complexity rank r(G), such that min_order(K) = Θ(r(G)).

Rationale (proposer's reasoning):

> Combinatorial design theory offers a rich toolkit for constructing and analyzing structures that can be used to understand the combinatorial properties of graphs. Kneser graphs, being highly connected structures, provide a useful framework for studying communication complexity. A strong correlation between the minimal order of a Kneser graph containing G and the communication complexity rank r(G) suggests a potential link between the structural complexity of the graph and its communication complexity.

Taxonomy category: `CombinatorialDesignTheory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ee65232e03a108a0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient (r) between min_order(K) and r(G) exceeds 0.7 for at least 80% of the 30 seeds, with no seed having a negative r-value. The conjecture is falsified if this condition is not met.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Kneser graph minimal order" AND "communication complexity rank"` - `"induced subgraph Kneser graph" related TO "communication complexity"` - `"correlation minimal order Kneser graph" WITH "communication complexity rank"`

Top relevant hits considered: - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2210.01601v2] Quantum communication complexity of linear regression - [http://arxiv.org/abs/2405.16881v1] Half-duplex communication complexity with adversary can be less than the classical communication complexity - [http://arxiv.org/abs/2305.05470v2] Active Personal Eye Lens Dosimetry with the Hybrid Pixelated Dosepix Detector - [http://arxiv.org/abs/hep-lat/0410024v1] The scaling dimension of low lying Dirac eigenmodes and of the topological charge density - [http://arxiv.org/abs/1112.1263v3] Rejection-free Monte-Carlo sampling for general potentials

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_rank(graph):
        # Placeholder for actual computation
        return len(graph)

    def is_subgraph(G, H):
        nodes_G = set(G.keys())
        nodes_H = set(H.keys())
        if not nodes_G.issubset(nodes_H):
            return False
        for u in nodes_G:
            for v in nodes_G:
                if (u, v) in G and (u, v) not in H:
                    return False
        return True

    def min_order_kneser(G):
        n = len(G)
        k = 1
        while True:
```

```
            K_n_k = {frozenset(range(i, i+k)) for i in range(n-k+1)}
            if any(is_subgraph(G, K) for K in K_n_k):
                return k
            k += 1

    def pearson_correlation(x, y):
        n = len(x)
        mean_x = sum(x) / n
        mean_y = sum(y) / n
        cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
        std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
        std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
        return cov / (std_x * std_y)

    results = []
    for _ in range(30):
        n = random.randint(5, 40)
        G = {i: set() for i in range(n)}
        edges = [(random.randint(0, n-1), random.randint(0, n-1)) for _ in range(random.randint(ir
        for u, v in edges:
            G[u].add(v)
            G[v].add(u)

        min_order = min_order_kneser(G)
        r_G = communication_rank(G)
        results.append((min_order, r_G))

    mean_min_order = sum(x[0] for x in results) / len(results)
    mean_r_G = sum(x[1] for x in results) / len(results)
    correlation = pearson_correlation([x[0] for x in results], [x[1] for x in results])

    return {
        "metric_name": "Pearson Correlation",
        "metric_value": correlation,
        "instances_tested": 30,
        "n_max": max(n for _, _ in results),
        "conjecture_holds": correlation > 0.7 and all(x >= 0 for x in [correlation]),
        "counterexample": "" if correlation > 0.7 else "Pearson Correlation < 0.7"
    }

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 4
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_corr = sum(x["metric_value"] for x in results) / len(results)
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr} std=0.0 support_fraction={support_fraction:.2f}
    else:
```

```
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"Pearson Correlation < 0.7\" first_failing_seed=
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eb4ad08.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eb4ad08.py", line 63, in run_trial
    min_order = min_order_kneser(G)
                ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eb4ad08.py", line 41, in min_order_kne
    if any(is_subgraph(G, K) for K in K_n_k):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eb4ad08.py", line 41, in <genexpr>
    if any(is_subgraph(G, K) for K in K_n_k):
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eb4ad08.py", line 27, in is_subgraph
    nodes_H = set(H.keys())
                  ^^^^^^
AttributeError: 'frozenset' object has no attribute 'keys'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to calculate the Pearson correlation coefficient and support fraction. | next: Investigate the cause of the crash in the test code and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14066
2	preregistration	ollama_remote	glm4:latest	0	0	9274
3	novelty	ollama_remote	glm4:latest	0	0	8768
4	novelty	ollama_remote	glm4:latest	0	0	13705

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24390
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11810
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22938
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17794
9	judge	ollama_remote	glm4:latest	0	0	9066

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 131811 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c2792048ca24.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c2792048ca24.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c2792048ca24.tar.gz` (if generated)